CHAPTER 3

REQUIREMENTS DETERMINATION

One of the first activities of an analyst is to determine the business requirements for a new system. This chapter begins by presenting how to analyze requirements using requirements analysis approaches and how to gather requirements using interviews, questionnaires, observation, and document analysis. The chapter next describes a set of approaches to perform text analysis on all of the material gathered. It then describes the requirements definition, along with user stories, as a way to document the new system's requirements. Finally, the chapter describes the system proposal document that pulls everything together.

OBJECTIVES

- Become familiar with requirements-analysis approaches.
- Understand when to use each requirements-analysis approach.
- Understand how to gather requirements using interviews, questionnaires, observation, and document analysis.
- Understand how sentence diagramming and ethnographic analysis can be used to uncover the actual system requirements.
- Understand how to create a requirements definition.
- Understand the use of user stories as an alternative to a formal requirements definition.
- Be able to begin creating a system proposal.

INTRODUCTION

The systems development process aids an organization in moving from the current system (often called the *as-is system*) to the new system (often called the *to-be system*). The output of planning, discussed in Chapter 2, is the system request, which provides general ideas for the to-be system, defines the project scope, and provides the initial workplan. Analysis takes the general ideas in the system request and refines them into a detailed requirements definition (this chapter), functional models (Chapter 4), structural models (Chapter 5), and behavioral models (Chapter 6) that together form the *system proposal*. The system proposal also includes revised project management deliverables, such as the feasibility analysis and the workplan (Chapter 2).

The output of analysis, the system proposal, is presented to the approval committee, which decides if the project is to continue. If approved, the system proposal moves into design, and its elements (requirements definition and functional, structural, and behavioral models) are used as inputs to the steps in design. This further refines them and defines in much more detail how the system will be built.

The line between analysis and design is very blurry. This is because the deliverables created during analysis are really the first step in the design of the new system. Many of the major

design decisions for the new system are found in the analysis deliverables. It is important to remember that the deliverables from analysis are really the first step in the design of the new system.

In many ways, because it is here that the major elements of the system first emerge, the requirements-determination step is the single most critical step of the entire systems development process. During requirements determination, the system is easy to change because little work has been done yet. As the system moves through the systems development process, it becomes harder and harder to return to requirements determination and to make major changes because of all of the rework that is involved. Several studies have shown that more than half of all system failures are due to problems with the requirements.[1] This is why the iterative approaches of object-oriented methodologies are so effective—small batches of requirements can be identified and implemented in incremental stages, allowing the overall system to evolve over time.

REQUIREMENTS DETERMINATION

The purpose of *requirements determination* is to turn the very high-level explanation of the business requirements stated in the system request into a more precise list of requirements that can be used as inputs to the rest of analysis (creating functional, structural, and behavioral models). This expansion of the requirements ultimately leads to the design of the system.

Defining a Requirement

A *requirement* is simply a statement of what the system must do or what characteristic it must have. During analysis, requirements are written from the perspective of the businessperson, and they focus on the "what" of the system. Because they focus on the needs of the business user, they are usually called *business requirements* (and sometimes user requirements). Later in design, business requirements evolve to become more technical, and they describe how the system will be implemented. Requirements in design are written from the developer's perspective, and they are usually called *system requirements*.

We want to stress that there is no black-and-white line dividing a business requirement and a system requirement—and some companies use the terms interchangeably. The important thing to remember is that a requirement is a statement of what the system must do, and requirements will change over time as the project moves from inception to elaboration to construction. Requirements evolve from detailed statements of the business capabilities that a system should have to detailed statements of the technical way the capabilities will be implemented in the new system.

Requirements can be either functional or nonfunctional in nature. A *functional requirement* relates directly to a process a system has to perform or information it needs to contain. For example, requirements stating that a system must have the ability to search for available inventory or to report actual and budgeted expenses are functional requirements. Functional requirements flow directly into the creation of functional, structural, and behavioral models that represent the functionality of the evolving system (see Chapters 4, 5, and 6). Functional requirements are essentially the business requirements uncovered as part of modeling the problem domain.

Nonfunctional requirements refer to behavioral properties that the system must have, such as performance and usability. The ability to access the system using a Web browser is

[1] For example, see *The Scope of Software Development Project Failures* (Dennis, MA: The Standish Group, 1995).

considered a nonfunctional requirement. Nonfunctional requirements can influence the rest of analysis (functional, structural, and behavioral models) but often do so only indirectly; nonfunctional requirements are used primarily in design when decisions are made about the database, the user interface, the hardware and software, and the system's underlying physical architecture.

Nonfunctional requirements describe a variety of characteristics regarding the system: operational, performance, security, and cultural and political. Operational requirements address issues related to the physical and technical requirements in which the system will operate. Performance requirements address issues related to the speed, capacity, and reliability of the system. Cultural and political requirements deal with issues related to the cultural and political factors and legal requirements that affect the system. Security requirements deal with issues with regard to who has access to the system, what type of access, and under what specific circumstances. In today's world of cyberwar, denial of service attacks, malware, phishing, ransomware, social engineering, spyware, Trojan horses, and many other types of security threats, the security requirements have become the most important of the nonfunctional requirements. Given that nonfunctional requirements do not describe business processes or information, in many cases, they have been ignored. However, if a system is to be deployed successfully, all of these requirements must be addressed. For pedagogical purposes, we delay addressing these requirements in detail until later in the book when we discuss design (see Chapters 8, 9, 10, and 11) and construction (see Chapter 12).

One area of information systems development that focused on differentiating functional and nonfunctional requirements is *software quality*. There have been many different models proposed to measure the quality of software. However, virtually all of them differentiate functional and nonfunctional requirements. From a quality perspective, *functional* quality is related to the degree that the software meets the functional requirements, that is, how much of the actual problem is solved by the software solution provided, whereas, the nonfunctional requirements are associated with the efficiency, maintainability, portability, reliability, reusability, testability, and usability quality dimensions.

When considering ISO 9000 compliance, quality dimensions are further decomposed into those that the user can see (*external*) and those that the user cannot see (*internal*). The external nonfunctional dimensions include efficiency, reliability, and usability, whereas the internal nonfunctional dimensions include maintainability, portability, reusability, and testability. From a user perspective, the external dimensions are more important. If the system is simply too difficult to use, regardless of how well the system solves the problem, the user will simply not use the system. In other words, from a user's perspective, for an information system to be successful, the system must not only meet the functional specification, but it must also meet the external nonfunctional specifications. From a developer's perspective, the internal dimensions are also important. For example, given that successful systems tend to be long-lived and multiplatform, both the maintainability and portability dimensions can have strategic implications for the system being developed. Also, given the *agile* development approaches being used in industry today, the development of reusable and testable software is crucial.

Three additional topics that have influenced information systems requirements are the Sarbanes–Oxley Act, COBIT (Control OBjectives for Information and related Technology) compliance, and Capability Maturity Model compliance. Depending on the system being considered, these three topics could affect the definition of a system's functional requirements, nonfunctional requirements, or both. The Sarbanes–Oxley Act, for example, mandates additional functional and nonfunctional requirements. These include additional security concerns (nonfunctional) and specific information requirements that management must now provide (functional). When developing financial information systems, information system

developers should be sure to include Sarbanes–Oxley expertise in the development team. Moreover, a client could insist on COBIT compliance or that a specific Capability Maturity Model level had been reached in order for the firm to be considered as a possible vendor to supply the system under consideration. Obviously, these types of requirements add to the nonfunctional requirements. Further discussion of these topics is beyond the scope of this book.[2]

Another recent topic that influences requirements for some systems is globalization. For example, a global information supply chain generates a large number of additional nonfunctional requirements. If the necessary operational environments do not exist for a mobile solution to be developed, it is important to adapt the solution to the local environment. Or, it may not be reasonable to expect to deploy a high-technology-based solution in an area that does not have the necessary power and communications infrastructure. In some cases, we may need to consider supporting some parts of the global information supply chain with manual—rather than automated—systems.

Manual systems have an entirely different set of requirements that create different performance expectations and additional security concerns. Furthermore, cultural and political concerns are potentially paramount. A simple example that affects the design of user interfaces is the proper use of color on forms (on a screen or paper). Different cultures interpret different colors differently. In other words, in a global, multicultural business environment, addressing cultural concerns goes well beyond simply having a multilingual user interface. We must be able to adapt the global solution to the local realities. Friedman refers to these concerns as glocalization.[3] Otherwise, we will simply create another example of a failed information systems development project.

Real-World Problems with Requirements Determination

Avison and Fitzgerald provide us with a set of problems that can arise with regard to determining the set of requirements with which to be dealt.[4] First, the analyst might not have access to the correct set of users to uncover the complete set of requirements. This can lead to requirements being missed, misrepresented, and/or over-specified. Second, the specification of the requirements may be inadequate. This can be especially true with the lightweight techniques associated with agile methodologies. Third, some requirements are simply unknowable at the beginning of a development process. However, as the system is developed, the users and analysts will get a better understanding of both the domain issues and the applicable technology. This can cause new functional and nonfunctional requirements to be identified and current requirements to evolve or be canceled. Iterative and incremental-based development methodologies, such as the Unified Process and agile, can help in this case. Fourth, verifying and validating of requirements can be very difficult. We take up this topic in the chapters that deal with the creation of functional (Chapter 4), structural (Chapter 5), and behavioral (Chapter 6) models.

[2] A concise discussion of the Sarbanes–Oxley Act is presented in G. P. Lander, *What Is Sarbanes-Oxley?* (New York: McGraw-Hill, 2004). A good reference for Sarbanes–Oxley Act-based security requirements is D. C. Brewer, *Security Controls for Sarbanes-Oxley Section 404 IT Compliance: Authorization, Authentication, and Access* (Indianapolis, IN: Wiley, 2006). For detailed information on COBIT, see www.isaca.org; for ISO 9000, see www.iso.org; and for details on the Capability Maturity Model, see www.sei.cmu.edu/cmmi/.

[3] T. L. Friedman, *The World Is Flat: A Brief History of the Twenty-First Century, Updated and Expanded Edition* (New York: Farrar, Straus, and Giroux, 2006).

[4] See D. Avison and G. Fitzgerald, *Information Systems Development: Methodologies, Techniques, & Tools*, 4th Ed. (London: McGraw-Hill, 2006).